

```
In [3]: import dataset
```

Project 2 Writeup

Section 1: Data Processing

Part A:

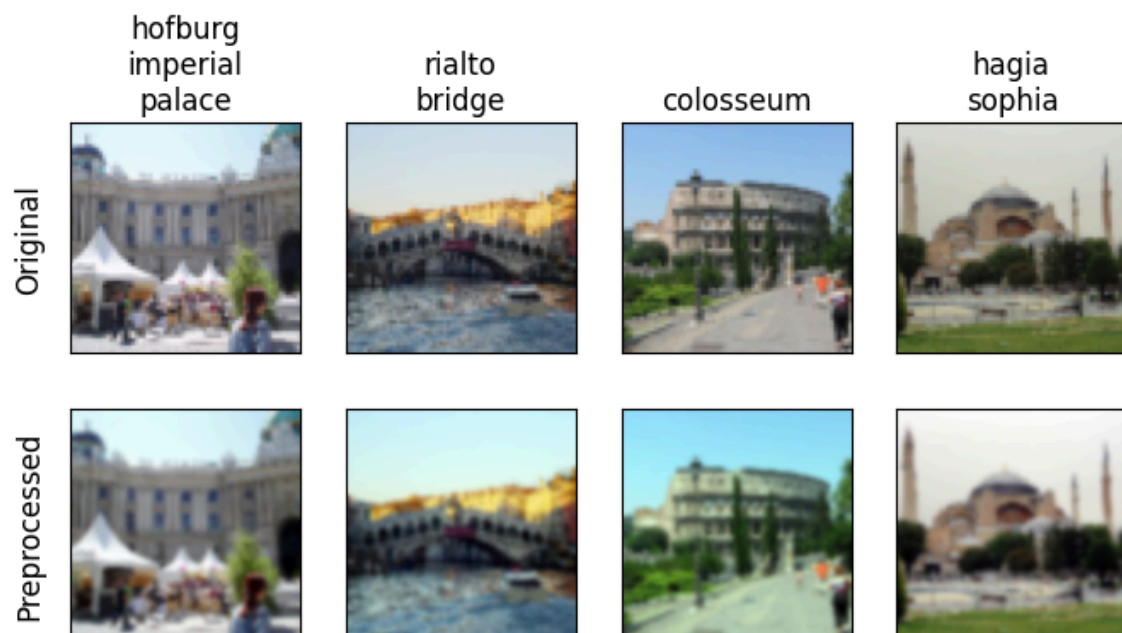
i.

Mean: [115.942 115.351 115.663]

Std: [63.827 66.688 72.982]

ii. A couple reasons. We calculate the mean and standard deviation from the training data to make sure that the dataset is normalized. When the model is trained on normalized data it ensures that the model training process is consistent, even if the data is scaled differently. This also makes sure that there is no data leakage between the training, testing, and validation processes. The model should not be aware of any of the data in the testing and validation sets to ensure that no biases creep into the training process and unfairly influence the final model.

Part B:



Section 2: Convolution Neural Networks

Part A:

General formula: $\text{kernel} * \text{number of filters}$

Layer 1: Convolutional Layer 1

$$\text{kernel} = 5 * 5 * 3 = 75, \text{filters} = 16 \rightarrow \text{kernel} * \text{filters} = 75 * 16 = 1200$$

Layer 2: Max Pooling Layer

0 parameters

Layer 3: Convolutional Layer 3

$$\text{kernel} = 5 * 5 * 16 = 400, \text{filters} = 64 \rightarrow \text{kernel} * \text{filters} = 400 * 64 = 25600$$

Layer 4: Max Pooling Layer

0 parameters

Layer 5: Convolutional Layer 3

$$\text{kernel} = 5 * 5 * 64 = 1600, \text{filters} = 8 \rightarrow \text{kernel} * \text{filters} = 1600 * 8 = 12800$$

Layer 6: Fully connected layer 1

$$\text{input} = 32, \text{output} = 2 \rightarrow \text{input} * \text{output} = 32 * 2 = 64$$

$$\text{parameters} = 1200 + 25600 + 12800 + 64 = 39664$$

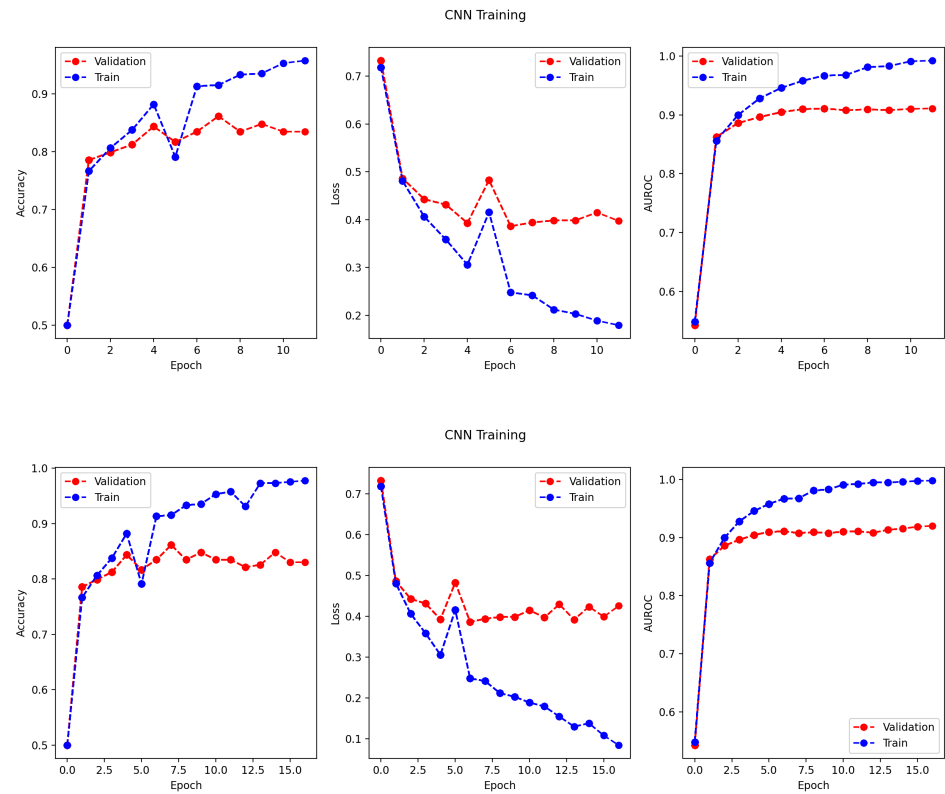
Part B-E:

see target.py, train_common.py, and train_cnn.py in code appendix

Part F:

i. One reason could be that the learning rate is not well suited for this model and it causes the weights to be incremented too fast. Maybe we accidentally overshoot the local minima, which could cause the spike in loss we see in the graph. Another reason could be that there is some sort of outlier in the training data and the model over corrects for that outlier, which causes the spike.

ii. With patience = 5, the model stops training at epoch 11. With patience = 10, the model stops training at epoch 16. I would say that patience = 5 is the better value since, looking at the graphs, the validation accuracy, loss, and AUROC scores start to stagnate after epoch 11 despite the training scores improving. This is a sign that the model is overfitting. A good reason to increase the patience value would be if the validation score show little signs of stagnation, meaning we haven't reached a minima yet. Or it could be increased to validate that we're not accidentally stuck in a local minima.



iii. The new size of the input to the fully connected layer would be 512 since the output of the last convolutional layer is $128 * 2 * 2$.

Filters	Epoch	Training AUROC	Validation AUROC
8 filters	6	around 0.96	around 0.9
128 filters	5	around 0.98	around 0.9

128 filters seems to prevent the loss from having random spikes compared to the loss with 8 filters. The performance of the model with 128 filters doesn't fluxate as much as with 8 filters. However, the additional filters did not improve the performance of the model very drastically.

Part G:

i.

	Training	Validation	Testing
AUROC	0.9668	0.9108	0.67

ii. The training and validation performances are fairly similar, which means the model should be properly fitted.

iii. The testing performance is noticably lower than the validation performance. This could be due to a few things. There could be some information leakage between the

training and validation data, which causes the high performance. The training and validation sets could not be reflective of the actual distribution of the data. Or, maybe the testing set is not an accurate representation of the whole data set. It could be that was data was just not split up properly. Etc.

Section 3: Visualizing what the CNN has learned

Part A:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j - \frac{dy^c}{dA_{ij}^k}$$

$$\alpha_1^1 = \frac{1}{16} \sum_i \sum_j - \frac{dy^1}{dA_{ij}^1 = \frac{1}{16} (-1+1-2-1-1+1+1+1+2+2)} = \frac{3}{16}$$

$$\alpha_2^1 = \frac{1}{16} \sum_i \sum_j - \frac{dy^1}{dA_{ij}^2 = \frac{1}{16} (1+2+2+2+2+1+1-1-2-1)} = \frac{7}{16}$$

$$L_{GradCam}^1 = ReLU(\sum_k \alpha_k^c A_k) = ReLU(\alpha_1^1 A^1 + \alpha_2^1 A^2) = ReLU(\frac{3}{16} A^1 + \frac{7}{16} A^2)$$

$$= ReLU\left(\begin{bmatrix} \frac{3}{16} & \frac{3}{16} & \frac{6}{16} & \frac{3}{16} \\ \frac{3}{16} & \frac{6}{16} & \frac{3}{16} & 0 \\ 0 & \frac{3}{16} & 0 & -\frac{3}{16} \\ 0 & \frac{3}{16} & -\frac{6}{16} & -\frac{6}{16} \end{bmatrix} + \begin{bmatrix} \frac{7}{16} & \frac{7}{16} & \frac{7}{16} & \frac{7}{16} \\ \frac{14}{16} & \frac{14}{16} & \frac{14}{16} & \frac{14}{16} \\ \frac{14}{16} & \frac{14}{16} & \frac{7}{16} & 0 \\ -\frac{7}{16} & -\frac{7}{16} & -\frac{7}{16} & 0 \end{bmatrix} \right)$$

$$= ReLU\left(\begin{bmatrix} \frac{10}{16} & \frac{10}{16} & \frac{13}{16} & \frac{10}{16} \\ \frac{17}{16} & \frac{20}{16} & \frac{17}{16} & \frac{14}{16} \\ \frac{14}{16} & \frac{17}{16} & \frac{7}{16} & -\frac{3}{16} \\ -\frac{7}{16} & -\frac{4}{16} & -\frac{13}{16} & -\frac{6}{16} \end{bmatrix} \right)$$

$$= \begin{bmatrix} \frac{10}{16} & \frac{10}{16} & \frac{13}{16} & \frac{10}{16} \\ \frac{17}{16} & \frac{20}{16} & \frac{17}{16} & \frac{14}{16} \\ \frac{14}{16} & \frac{17}{16} & \frac{7}{16} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Part B:

Most images of the Hofburg Imperial Palace take place during the day with brighter colors and include a good portion of the sky in the image. It seems like the model is relying the most on these two features to identify the palace rather than the building itself. This means that if there are bright colors on people's clothing or sky in the

pictures of the Pantheon, the model might still identify the palace in those pictures when the palace is not there.

Part C:

This explains the low testing performance from 2(g) since the model is relying on the time of day and sky the most when identifying the Hofburg Imperial Palace vs. the Pantheon. If photos of the Pantheon in the testing data have more instances of sky or bright colors from lights, people's clothing, etc., then it would completely mess with the model's ability to make accurate predictions.

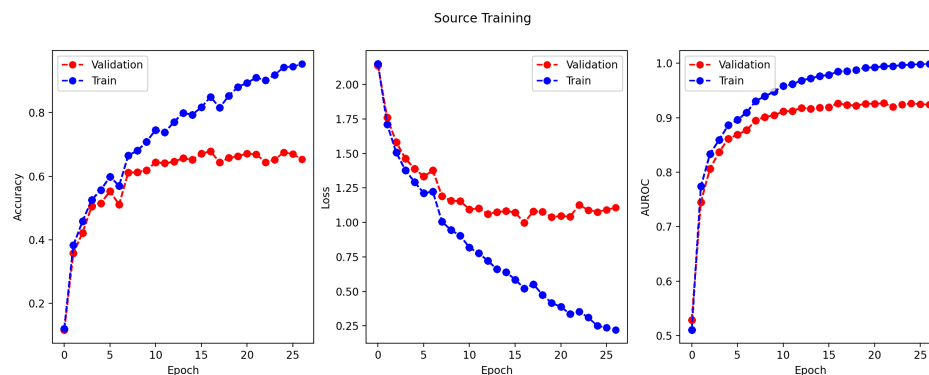
Section 4: Transfer Learning & Data Augmentation

4.1 Transfer Learning

Part A-B:

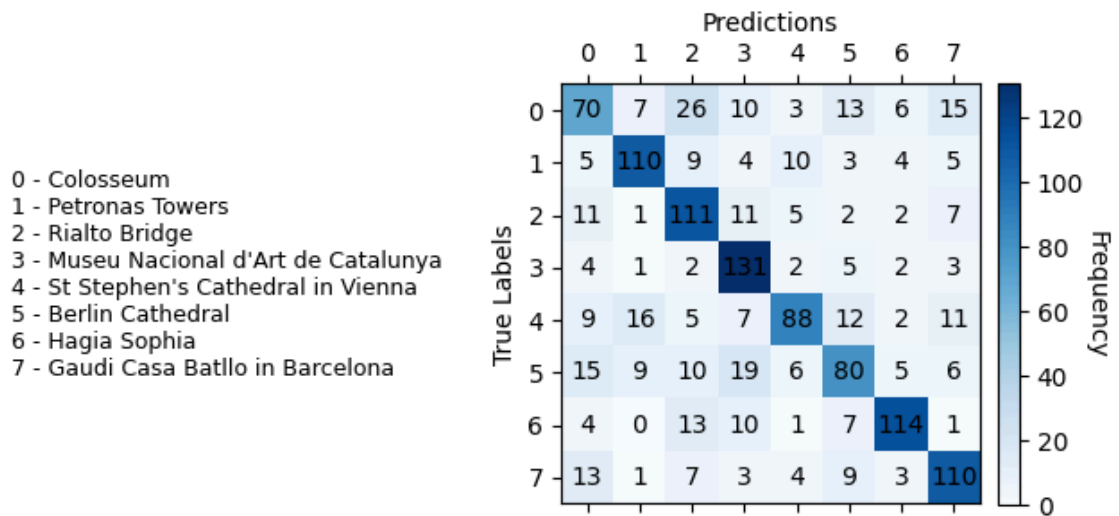
see source.py and train_source.py in code appendix

Part C:



Lowest validation loss around epoch 16.

Part D:



The classifier is most accurate for the Petronas Towers, Rialto Bridge, Museu Nacional d'Art de Catalunya, Hagia Sophia, and Gaudi Casa Batllo in Barcelona. Museu Nacional d'Art de Catalunya is the most accurate overall while Petronas Towers and Gaudi Casa Batllo in Barcelona are less accurate out of the other in this batch.

The classifier is least accurate for the Colosseum, St. Stephen's Cathedral in Vienna, and Berlin Cathedral. St. Stephen's Cathedral in Vienna is the most accurate in this batch and the Colosseum is the least accurate overall.

One thing I noticed is that the landmarks with the most accurate predictions (Museu Nacional d'Art de Catalunya, Rialto Bridge and Hagia Sophia) are pictured with a fountain or some body of water. This makes these landmarks much more distinguishable compared to the others. The other landmarks feature some kind of blue in their pictures. This also makes them distinguishable, but less so than the landmarks with water. In the case of the Colosseum, since it has blue similar to the Gaudi Casa Batllo in Barcelona pictures and the other colors are really similar to the colors in the Rialto Bridge pictures, it gets misclassified at a much higher rate.

Part E:

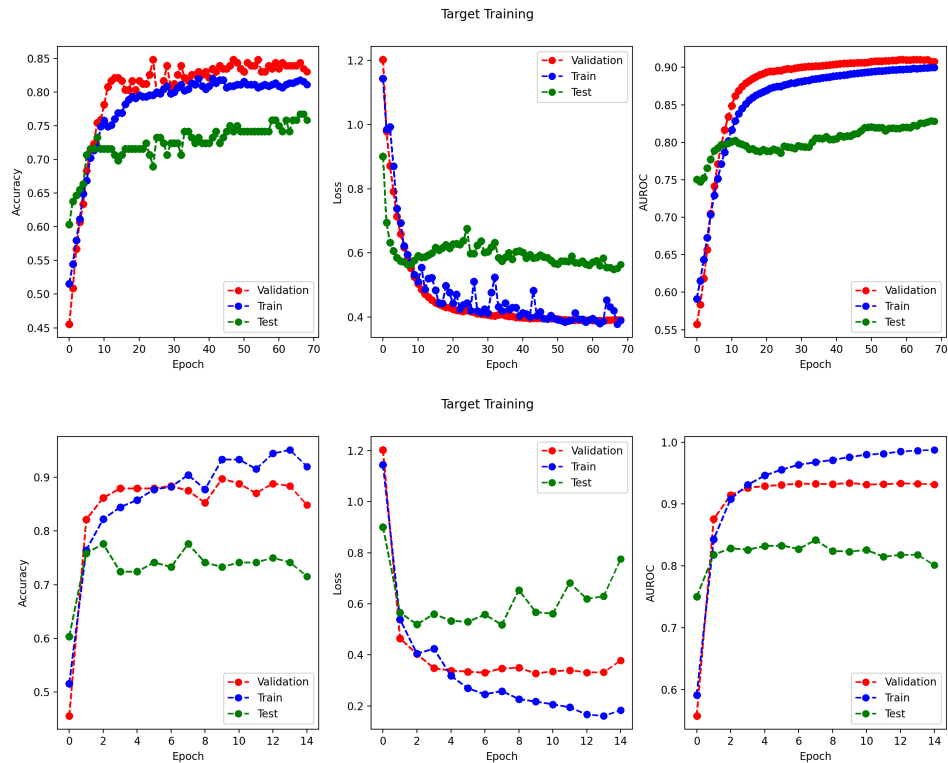
see train_target.py in code appendix

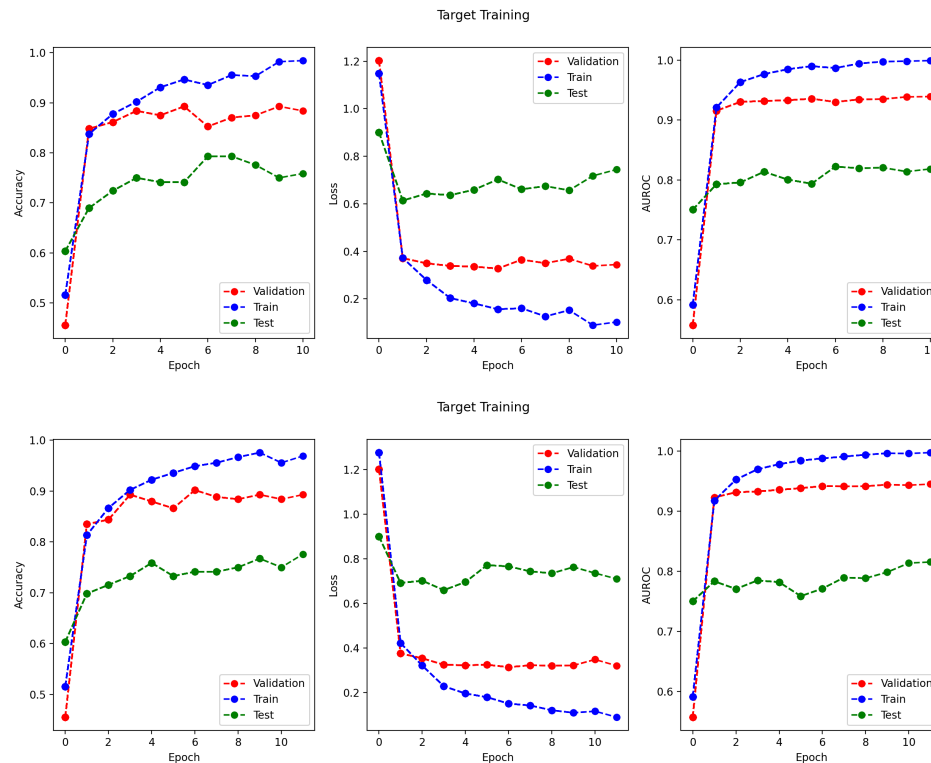
Part F:

AUROC scores

	Train	Val	Test
Freeze all CONV layers (Fine-tune FC layer)	0.8984	0.9101	0.8219
Freeze first two CONV layers (Fine-tune last CONV and FC layers)	0.976	0.9342	0.8231
Freeze first CONV layer (Fine-tune last 2 conv. and fc layers)	0.99	0.9357	0.7937
Freeze no layers (Fine-tune all layers)	0.9882	0.942	0.7714
No Pretraining or Transfer Learning (Section 2(g) performance)	0.9668	0.9108	0.67

(same order as table)





Overall, the source task was helpful to the target task, given that the testing performance is better over all for all the classifiers that used the source task. The testing performance for the model with no transfer learning was about 0.67 while the models with transfer learning have testing performances that range from 0.77 to 0.82, which is much better.

The general trend I see is that the testing performance for the transfer learning models goes down as we freeze more layers. This may be due to the fact that the model with more layers to fine-tune means that the model is able to account for the variance seen in the data but also starts to overfit more. This leads to a model that is less accurate during predictions because it has overfit to the training data.

4.2 Data Augmentation

Part A:

see `augment_data.py` in code appendix

Part B:

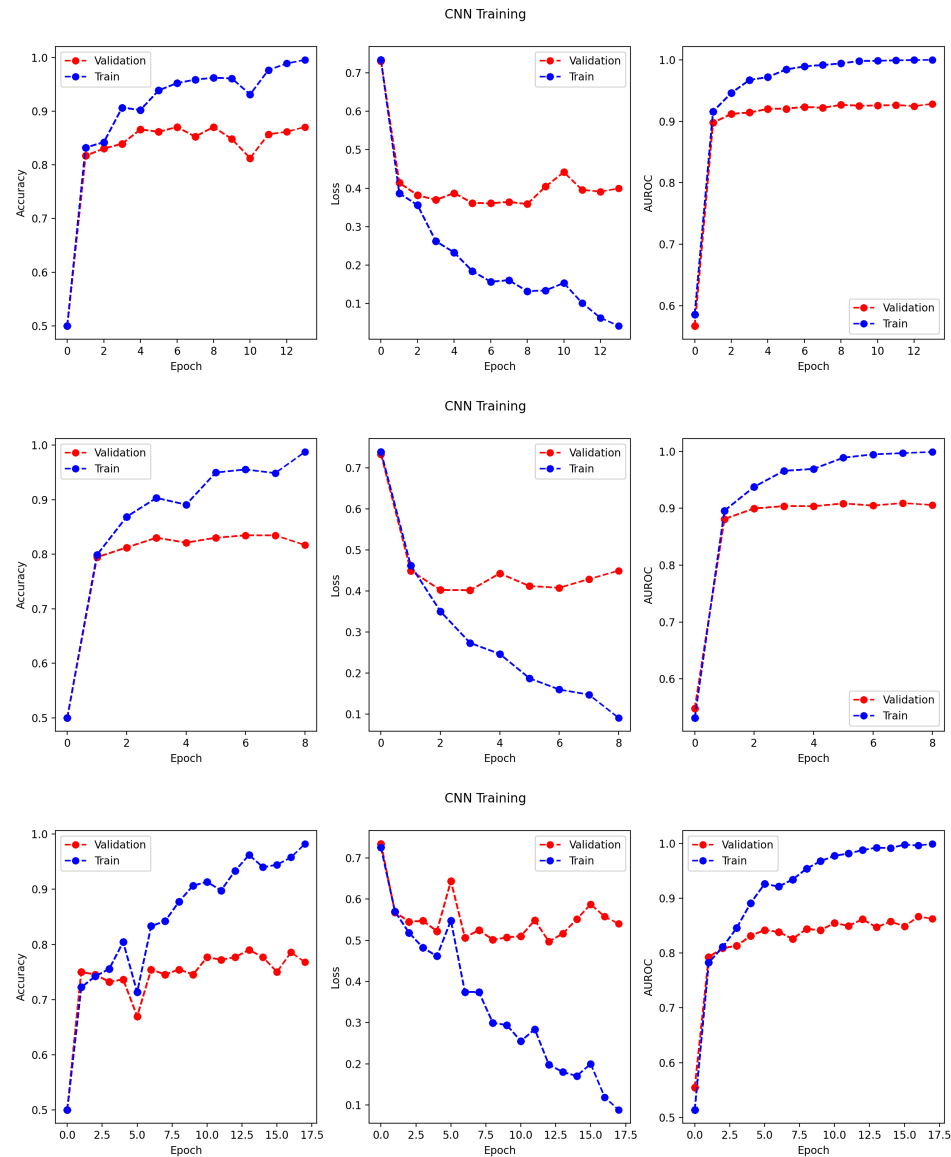
i-ii. see `augment_data.py` in code appendix

iii.

AUROC scores

	Train	Val	Test
Rotation (keep original)	0.995	0.929	0.7099
Grayscale (keep original)	0.9772	0.9054	0.7274
Grayscale (discard original)	0.9866	0.8609	0.8109
No augmentation (Section 2(g) performance)	0.9668	0.9108	0.67

(same order as table)



Part C:

Generally, the validation performances get worse when you go from rotation to grayscale with original to greyscale without original. The training performance is best for the rotation and worst for grayscale with original. I think the reason the validation performance goes down between rotation -> grayscale with original -> grayscale without original is because the model has less and less features to train off of with each change.

Therefore, the model becomes less flexible and it less able to properly predict the test data labels. The rotation training performance does really well since we have many features to train off of as well as the original images along with the rotated images. This means the model is better at fitting the training data and able to fine-tune itself a lot more than the other models. The grayscale without original also does well because there is less variance in the data when everything is in grayscale. I think this is why the grayscale with original does slightly worse, since there's more variance in the data with pictures in color and pictures in greyscale.

Section 5: Challenge

The general goal is the pull from previous models that maximized testing performance. This seems like a sound way to go about the challenge model, since we're being graded on how well we can predict labels in the testing data.

Regularization: I decided to go with the same regularization as used in the previous target and source models. They seem to have worked well for the training data, so I didn't see a reason to change them.

Feature selection: I decided to to augment the data with grayscale (discard original) so the model would focus more on the textures and the forms in the images rather than the colors. I don't want the model to rely on color since it generally worsens the testing performance.

Model architecture: The model architecture is pretty much untouched from the architectures in target.py. I don't see anything wrong with this architecture. The model didn't improve very much when I messed around with the number of output and input channels, so I left it be.

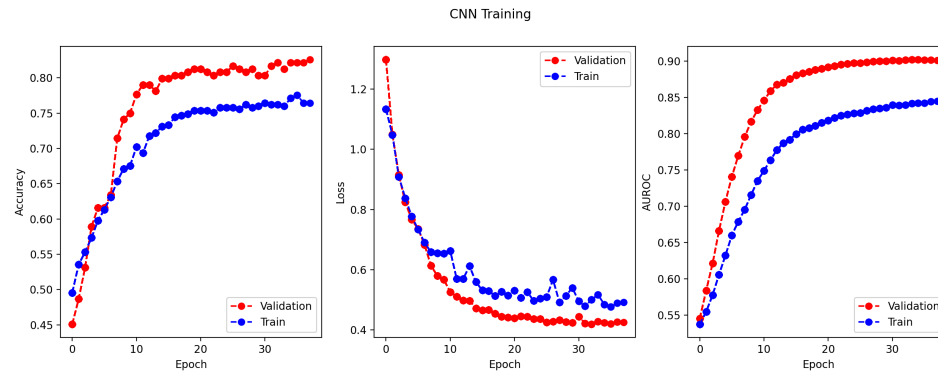
Hyperparameters: I also left the hyperparameters alone. Changing the hyperparameters didn't help my model very much so I left them alone as well.

Transfer learning: Since the testing performance improved with transfer learning. I decided to use the source model yielded by train_source.py since it worked pretty well previously. Also decided to freeze three layers and only fine tune the FC layer since, even though it had the worst training performance, it had the best test performance out of the other methods.

Data Augmentation: I decided to use grayscale (discard original) to augment the data since it yielded the best testing performance as well.

Model evaluation: I evaluated my model using the AUROC score and Validation loss. I don't necessary need to minimize these values but I want them to be around the range seen in the grayscale model and the FC fine-tuning model. I also want to see that my

validation scores are better than my training scores, since that increases my confidence that the model can do well on the training data.



see challange.py and train_challange.py in code appendix